# CO4_AT3 - Comparative Analysis

**Name: C. Himaja**

**Reg. No: 192373035**

## Source Code

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

// Function to solve Coin Change using Greedy method
int greedyCoinChange(int coins[], int n, int amount, int selected[]) {
    int count = 0;
    int remaining = amount;

    // Start from the largest denomination
    for (int i = n - 1; i >= 0; i--) {
        while (remaining >= coins[i]) {
            remaining -= coins[i];
            selected[i]++;
            count++;
        }
    }

    // If amount cannot be formed
    if (remaining != 0)
        return -1;
```

```
    return count;
}

// Function to solve Coin Change using Dynamic Programming
int dpCoinChange(int coins[], int n, int amount, int dp[], int used[]) {

    // Initialize DP array
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        dp[i] = INT_MAX;
        used[i] = -1;
    }

    // Build DP table
    for (int i = 1; i <= amount; i++) {

        for (int j = 0; j < n; j++) {

            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {

                if (dp[i - coins[j]] + 1 < dp[i]) {
                    dp[i] = dp[i - coins[j]] + 1;
                    used[i] = coins[j];
                }
            }
        }
    }

    return dp[amount];
```

```
}

// Display coins used
void displayCoins(int coins[], int n, int selected[]) {

    for (int i = 0; i < n; i++) {
        if (selected[i] > 0) {
            printf("%d x %d coin(s)\n",
                   selected[i],
                   coins[i]);
        }
    }
}

int main() {

    int coins[] = {1, 4, 6};
    int n = 3;
    int amount = 8;

    int greedySelected[MAX] = {0};

    int dp[MAX + 1];
    int used[MAX + 1];

    // ------------------------------
    // GREEDY METHOD
    // ------------------------------

    int greedyResult =
```

```
    greedyCoinChange(
        coins,
        n,
        amount,
        greedySelected
    );

// ------------------------------
// DYNAMIC PROGRAMMING
// ------------------------------

int dpResult =
    dpCoinChange(
        coins,
        n,
        amount,
        dp,
        used
    );

// ------------------------------
// OUTPUT
// ------------------------------

printf("\n==========================================\n");
printf(" COIN CHANGE PROBLEM COMPARISON\n");
printf("==========================================\n");

printf("\nCoin Denominations: [1, 4, 6]");
printf("\nAmount: %d\n", amount);
```

```
// Greedy result
printf("\n---------- GREEDY METHOD ----------\n");

if (greedyResult != -1) {

    printf("Coins selected:\n");
    displayCoins(
        coins,
        n,
        greedySelected
    );

    printf("Total number of coins = %d\n",
           greedyResult);
}
else {
    printf("Amount cannot be formed using Greedy method.\n");
}

// DP result
printf("\n------ DYNAMIC PROGRAMMING METHOD ------\n");

if (dpResult != INT_MAX) {

    printf("Coins selected: ");

    int current = amount;

    while (current > 0) {
```

```
        printf("%d ", used[current]);
        current -= used[current];
    }

    printf("\nTotal number of coins = %d\n",
           dpResult);
}
else {
    printf("Amount cannot be formed using DP.\n");
}

// Comparison
printf("\n---------- COMPARISON ----------\n");

printf("Greedy Coin Count = %d\n",
       greedyResult);

printf("DP Coin Count     = %d\n",
       dpResult);

if (greedyResult == dpResult) {
    printf("Greedy gives the optimal solution for this input.\n");
}
else {
    printf("Greedy does NOT give the optimal solution.\n");
    printf("Dynamic Programming gives the optimal solution.\n");
}

printf("\n========================================\n");
```

```
    return 0;
}
```